

Semoss VSCode Extension - User Manual

Version: 2.0.0

Last Updated: January 6, 2026

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Getting Started](#)
 4. [Features](#)
 5. [Chatbot Interface](#)
 6. [Instance Management](#)
 7. [MCP Server Configuration](#)
 8. [Deployment Workflows](#)
 9. [Troubleshooting](#)
 10. [FAQ](#)
-

Introduction

The Semoss VSCode Extension is a powerful tool designed to streamline your development workflow with Semoss applications. It provides:

- **Asset Management:** Zip and deploy Semoss applications directly from VSCode
 - **Interactive Chatbot:** AI-powered assistant built with React for enhanced support
 - **Multi-Instance Support:** Manage multiple Semoss instances with custom aliases
 - **MCP Integration:** Connect to external tools and services using Model Context Protocol
-

Installation

Prerequisites

- Visual Studio Code (version 1.60.0 or higher)
- Node.js (version 14.x or higher)
- Access to a Semoss instance

Install from VSCode Marketplace

1. Open VSCode
2. Go to Extensions (Ctrl+Shift+X / Cmd+Shift+X)
3. Search for "Semoss"
4. Click "Install"

Install from VSIX

1. Download the **.vsix** file
 2. Open VSCode
 3. Go to Extensions
 4. Click "... " menu → "Install from VSIX"
 5. Select the downloaded file
-

Getting Started

First-Time Setup

Step 1: Authorize Your First Instance

1. Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
2. Type and select: **"Semoss: Authorize New Instance"**
3. Enter an alias for your instance (e.g., "Production", "Development", "Staging")

Step 2: Generate Access Keys

1. Log in to your Semoss instance
2. Navigate to **Settings** → **My Profile** → **Generate**
3. Generate an **Access Key** and **Private Key**
4. Copy both keys for the next step

Step 3: Complete Authorization

1. Enter your Semoss URL (without trailing slash)
 - ☒ Correct: <https://workshop.cfg.deloitte.com/cfg-ai-demo>
 - ☐ Incorrect: <https://workshop.cfg.deloitte.com/cfg-ai-demo/>
 2. Paste the **Access Key** when prompted
 3. Paste the **Private Key** when prompted
 4. Confirmation message will appear once authorized
-

Features

Modern Chatbot Interface

The extension includes a sophisticated React-based chatbot that provides:

- **Interactive Assistance:** Get help with Semoss operations and commands
- **Modern UI:** Responsive design optimized for various screen sizes
- **Model Management:** Configure and switch between AI models
- **MCP Tools:** Access external tools and services

Access the Chatbot:

- Click the **Semoss Assistant** icon in the Activity Bar
- Or use Command Palette: **"Semoss: Open Chatbot"**

Instance Management

Manage multiple Semoss environments with ease:

- **Add Instances:** Authorize new instances with custom aliases
- **Switch Instances:** Quickly change between saved instances
- **Remove Instances:** Delete instances you no longer need
- **Persistent Storage:** All instances are saved across VSCode sessions

Asset Deployment

Right-click any of these folders in your workspace to deploy:

- `client/` - Client-side applications
- `py/` - Python backends
- `portals/` - Semoss portals

Deployment Options:

- **Zip and Deploy:** Create archive and deploy in one action
 - **Zip Only:** Create a zip archive without deploying
 - **Deploy Only:** Deploy an existing zip file
-

Chatbot Interface

Opening the Chatbot

1. Click the **Semoss Assistant** icon in the Activity Bar (left sidebar)
2. Or use Command Palette: `Semoss: Open Chatbot`

Using the Chat

1. Type your message in the input field at the bottom
2. Press **Enter** or click the **Send** button
3. The AI assistant will respond with helpful information

Settings Panel

Access settings by clicking the  **gear icon** in the chatbot header.

Configuration Tab

Edit your YAML configuration directly:

1. Make changes in the editor
2. Click **"Save Configuration"**
3. Status banner appears:
 -  "Saving configuration..." (in progress)
 -  "Configuration saved." (success)
 -  Error message (if validation fails)

MCP Servers Tab

Configure Model Context Protocol servers to connect external tools. See [MCP Server Configuration](#) section for detailed instructions.

Instance Management

Adding a New Instance

1. Command Palette → **"Semoss: Authorize New Instance"**
2. Enter an alias (e.g., "QA Server", "Client Demo")
3. Enter the Semoss URL (without trailing slash)
4. Provide Access Key and Private Key from Semoss

Switching Between Instances

1. Command Palette → **"Semoss: Select Instance"**
2. Select from the list of saved instances
3. All deployments will now use the selected instance

Removing an Instance

1. Command Palette → **"Semoss: Remove Instance"**
2. Select the instance to remove
3. Confirm deletion

Note: This only removes the instance from VSCode storage. It does not affect the actual Semoss instance.

MCP Server Configuration

What is MCP?

The Model Context Protocol (MCP) allows the chatbot to connect to external tools and services, extending its capabilities beyond the base AI model.

Accessing MCP Manager

1. Open the chatbot
2. Click the  **settings gear icon**
3. Navigate to the **"MCP Servers"** tab

Adding a Custom MCP Server

Step 1: Click "+ Add Custom Server"

Step 2: Fill in Server Details

Required Fields:

- **Server Name:** Unique identifier (e.g., "filesystem", "docker", "git")
- **Command:** Executable command
 - Examples: `npx`, `node`, `python`, `docker`
 - Windows: Use full path for npx: `C:\Program Files\nodejs\npx.cmd`

Optional Fields:

- **Arguments:** Space-separated command arguments
 - Example: `@modelcontextprotocol/server-git`
- **Description:** Brief description of the server's purpose
- **Timeout:** Maximum execution time in milliseconds (default: 15000)
 - Applies to all tools on this server
- **URL:** API endpoint for HTTP-based servers
 - Example: `http://localhost:9090/Monolith/api/ext/mcp/appid/comms`
- **Method:** HTTP method (GET, POST, PUT, DELETE)
- **HTTP Headers:** Key-value pairs for authentication
 - Click "+ Add Header" to add multiple headers
 - Example: `Authorization: Bearer your-token-here`

Step 3: Configure Tools

Each server must have at least one tool defined:

- **Tool Name:** Identifier for the tool (e.g., "Date", "SearchFiles")
- **Command:** Command to execute
 - For HTTP servers: use `semoss_npx`
 - For local tools: specify the command
- **Description:** What the tool does
- **Parameters:** (Optional) JSON schema defining tool inputs

Parameters Example:

```
{
  "type": "object",
  "properties": {
    "format": {
      "type": "string",
      "description": "Date format (e.g., 'DD-MM-YYYY')"
    },
    "date": {
      "type": "string",
      "description": "Date string (e.g., '2023-03-15')"
    }
  },
  "required": []
}
```

Step 4: Save the Server

Click "**Add Server**" to save your configuration.

Configuration Examples

Example 1: NPX-based Git Server

```
mcpServers:
  - name: git
    command: npx
    args:
      - '@modelcontextprotocol/server-git'
    timeout: 5000
    enabled: true
    tools: []
```

Example 2: HTTP-based Semoss Date Tool

```
mcpServers:
  - name: Date
    command: npx
    args: []
    timeout: 15000
    enabled: true
    url: http://localhost:9090/Monolith/api/ext/mcp/your-id/comms
    method: POST
    headers:
      Content-Type: application/json
      Authorization: Bearer your-token-here
    tools:
      - name: Date
        command: semoss_npx
        description: Get current date and time from your Semoss platform
        parameters:
          type: object
          properties:
            format:
              type: string
              description: Date format (e.g., 'DD-MM-YYYY', 'YYYY-MM-DD')
            date:
              type: string
              description: Date string (e.g., '2023-03-15')
          required: []
```

Key Points:

- `url`, `method`, `headers` are defined at the **server level**
- `timeout` applies to all tools on this server
- `command: semoss_npx` identifies this as an HTTP tool

- **parameters** defines the JSON schema for tool inputs
- Replace **your-id** and **your-token-here** with actual values

Managing MCP Servers

Enable/Disable:

- Toggle the ON/OFF switch on each server card
- Disabled servers won't be loaded by the chatbot

Edit Server:

1. Click the  **Edit** button
2. Modify any fields
3. Click "**Update Server**"

Delete Server:

1. Click the  **Delete** button
2. Confirm deletion in the popup

Add Tools to Existing Server:

1. Edit the server
 2. Scroll to Tools Configuration
 3. Click "+ **Add Tool**"
 4. Fill in tool details
 5. Click "**Update Server**"
-

Deployment Workflows

Deploying Assets

Method 1: Right-Click Menu

1. In VSCode Explorer, right-click on:
 - **client/** folder
 - **py/** folder
 - **portals/** folder
2. Select one of:
 - **Semoss: Zip and Deploy** - Creates zip and deploys
 - **Semoss: Zip Only** - Creates zip archive
 - **Semoss: Deploy Only** - Deploys existing zip

Method 2: Command Palette

1. Open Command Palette (Ctrl+Shift+P / Cmd+Shift+P)
2. Type "Semoss" to see available commands

3. Select the desired deployment command

Deployment Process

1. **Zipping:** Files are archived with proper structure
 2. **Uploading:** Archive is sent to the selected Semoss instance
 3. **Confirmation:** Success/error message appears
 4. **Status Bar:** Shows deployment progress
-

Troubleshooting

Common Issues

"spawn npx ENOENT" Error

Problem: Windows cannot find the npx command.

Solution:

- Use full path to npx.cmd:

```
command: "C:\\Program Files\\nodejs\\npx.cmd"
```

- Verify Node.js is installed: Open terminal and run `node --version`
- Check if npx is in your PATH

MCP Tool Not Responding

Problem: Tool execution times out or fails.

Solutions:

1. **Increase Timeout:** Raise the timeout value (e.g., from 15000 to 30000)
2. **Check URL:** Verify the URL is accessible
 - Test in browser or with curl/Postman
3. **Verify Headers:** Ensure Authorization headers contain valid tokens
 - Check if token has expired
4. **Check Logs:** Look at the chatbot console for error messages

Server Not Showing Tools

Problem: MCP server is added but tools don't appear.

Solutions:

1. Confirm `enabled: true` is set on the server
2. Verify tools array is properly configured with at least one tool
3. Check the tool `command` field (should be `semoss_npx` for HTTP servers)

4. Reload the chatbot window (close and reopen)

Configuration Not Saving

Problem: YAML edits don't persist after saving.

Solutions:

1. Check for YAML syntax errors (indentation, colons, quotes)
2. Ensure the config directory has write permissions
3. Look for validation error messages in the status banner
4. Try "Reset to Defaults" then re-enter your configuration

Authorization Fails

Problem: Cannot authorize a new Semoss instance.

Solutions:

1. Verify the URL format (no trailing slash)
 2. Regenerate Access Key and Private Key from Semoss
 3. Check if your Semoss instance is accessible (ping the URL)
 4. Ensure you're copying the full keys without extra spaces
-

FAQ

General Questions

Q: What is the difference between client, py, and portals folders?

A:

- `client/` - Contains frontend JavaScript/React applications
- `py/` - Contains Python backend services and scripts
- `portals/` - Contains Semoss portal configurations

Q: Can I deploy to multiple Semoss instances simultaneously?

A: No, deployments go to the currently selected instance. Switch instances using "Semoss: Select Instance" before deploying.

Q: Where is my configuration stored?

A: The YAML configuration is stored in `config/semoss-config.yaml` within the extension directory.

MCP Questions

Q: What types of MCP servers can I add?

A: Two types:

1. **Local Process Servers:** Run as separate processes (npx, node, python, etc.)

2. HTTP-based Servers: Connect to REST APIs (Semoss tools, custom services)

Q: Can I use environment variables in my configuration?

A: Yes, use placeholders like `$OPENAI_API_KEY` in your YAML. Ensure they're set in your environment before VSCode starts.

Q: How do I debug MCP tool failures?

A:

1. Check the Output panel (View → Output)
2. Select "Semoss Extension" from the dropdown
3. Look for error messages and stack traces

Q: Can MCP servers access my local filesystem?

A: Only if you configure a filesystem server with appropriate permissions. By default, MCP servers are sandboxed.

Chatbot Questions

Q: Can the chatbot access my code?

A: The chatbot can see context you provide and use configured MCP tools. It follows VSCode's security model.

Q: How do I change the AI model?

A: Go to Settings → Model Manager, add your preferred model, and select it from the model dropdown in the chat.

Q: Does the chatbot work offline?

A: The UI works offline, but AI responses require an internet connection to reach the model provider's API.

Version Information

- **Current Version:** 2.0.0
- **Minimum VSCode Version:** 1.60.0
- **Node.js Version:** 14.x or higher

Appendix

YAML Configuration Reference

```
models:
  - name: string           # Model identifier
    provider: string       # API provider
    model: string          # Model version/ID
    apiKey: string         # Authentication key
```

```
    baseUrl: string      # Optional API endpoint
    temperature: number   # 0.0 to 1.0
    maxTokens: number     # Maximum response length
    enabled: boolean      # Whether model is active

mcpServers:
  - name: string          # Server identifier
    command: string       # Executable command
    args: string[]        # Command arguments
    timeout: number       # Milliseconds (server-level)
    enabled: boolean      # Whether server is active
    url: string           # API endpoint (optional)
    method: string        # HTTP method (optional)
    headers:              # HTTP headers (optional)
      key: value
    tools:
      - name: string      # Tool identifier
        command: string   # Tool command
        description: string
        parameters:       # JSON schema (optional)
          type: object
          properties: {}
          required: []
```

Glossary

- **MCP**: Model Context Protocol - Standard for connecting AI models to external tools
- **NPX**: Node Package eXecute - Runs Node.js packages without global installation
- **YAML**: Human-readable data serialization format
- **Access Key**: Public identifier for Semoss authentication
- **Private Key**: Secret key for Semoss authentication
- **Alias**: Custom name for a Semoss instance

Document Version: 1.0

Last Updated: January 6, 2026

Prepared by: Semoss Team